Flow Matching Neural Processes: Reimplementation, Calibration and Sampling Cost

Samuel Déniz, Carlos Enrique Pereda and Yaser Jafar
Universidad Autónoma de Madrid

Abstract

Flow Matching Neural Processes (FlowNP) model conditional distributions over functions by learning a velocity field with a transformer, and report state-of-the-art likelihoods on standard neural-process benchmarks. We reproduce the Gaussian-process benchmark of the original paper with the official code (three kernels, five models), write an independent reimplementation that is numerically equivalent to the official one (same velocities up to $2{\cdot}10^{-6}$ and the same test likelihood, $1.712{\pm}0.003$ against $1.716{\pm}0.004$), and study three aspects the paper does not evaluate. (i) *Calibration*: FlowNP samples are mildly over-confident (83% coverage of nominal 90% intervals), and the stochastic sampler shipped with the official code is severely over-confident (60%). (ii) *Sampling cost*: a fourth-order Runge–Kutta solver with 5 steps matches or exceeds the calibration of the 100-step Euler sampler at about one fifth of the cost, while Euler strongly over-estimates likelihoods. (iii) *Multimodality*: on a synthetic family of cosines whose exact posterior is available in closed form, FlowNP produces the most faithful samples when the context is informative (98% valid curves with 20 points), but trained from scratch it stalls in a plateau where it ignores the context; a Gaussian-process initialisation removes the problem. With very few points, a diffusion model with guidance covers the posterior modes better. All code, tests, a tutorial and the experiment logs are released.

1 Introduction

Many prediction problems involve inferring an unknown function from a limited number of observations. Examples include interpolating sensor readings, fitting a surrogate model, and completing in parts of an image. A *neural process* (NP) addresses this challenge by meta-learning across a family of functions. Given a context set $C = \{(x_i, y_i)\}_{i=1}^{M}$ observed from a single function, the NP outputs the predictive distribution $p\theta(y^{\mathrm{tar}} \mid x^{\mathrm{tar}}, C)$ for an arbitrary set of N target inputs in a single forward pass, without requiring per-function training [3, 4]. Since the model produces a distribution rather than only a mean, its effectiveness depends on how accurately this distribution represents uncertainty about the function.

Successive NP families trade expressiveness against cost. Conditional NPs factorise the prediction over targets into independent Gaussians and tend to underfit [3, 7]. Transformer NPs with an autoregressive decoder are more expressive but generate one point at a time with a Gaussian conditional at every step [12]. Neural diffusion processes model non-Gaussian marginals but need auxiliary conditioning mechanisms [2]. Abu Hamad and Rosenbaum [1] propose FlowNP, which combines the NP framework with flow matching [9, 10]: a transformer reads the context points and the noisy target values at time t and predicts their velocity, so that conditional samples are obtained by integrating an ordinary differential

equation (ODE) for all targets in parallel, and exact likelihoods by integrating it backwards with a Hutchinson trace estimator [5, 6]. They report higher likelihood than every baseline.

The study leaves three questions open, which this report addresses.

- **Calibration.** Likelihood is one scalar summary of a predictive distribution. It does not say whether a 90% interval contains the truth 90% of the time, nor whether the model is over- or under-confident.
- **Cost.** Sampling requires several network evaluations whose number depends on the solver and the step count. The original analysis of this trade-off is partial, so it is unclear how few steps suffice and whether higher-order solvers pay off.
- **Sample quality against a ground truth.** A high likelihood does not guarantee that samples are valid functions, nor that they cover all the explanations compatible with the context. Testing this requires a task whose posterior is known and non-Gaussian.

Our contributions are:

1. an independent, tested PyTorch reimplementation of FlowNP (training, sampling and likelihood) that can load official checkpoints, which provides a direct check that both architectures coincide;
2. a calibration and cost evaluation protocol for NPs, with interval coverage and CRPS, and a solver $\times$ steps sweep reported against network evaluations and wall-clock time;
3. a new synthetic benchmark of noisy cosines with a closed-form four-component Gaussian-mixture posterior, which gives an exact reference likelihood and fidelity and diversity metrics for samples;
4. a public repository with unit tests, pinned reference code, versioned configurations and an executable tutorial.

2 Objectives

General objective. To reimplement FlowNP independently and to characterise, with a reproducible protocol, the quality of its uncertainty and its accuracy–cost trade-off with respect to earlier neural processes.

Specific objectives.

O1 Reimplement FlowNP in PyTorch and validate it against the official implementation and published figures on one-dimensional Gaussian processes.

O2 Reproduce the comparison with the conditional (CNP), attentive (ANP), autoregressive-transformer (TNPA) and diffusion (NDP) neural processes, and with the official FlowNP, on Gaussian-process data.

O3 Extend the evaluation with calibration (interval coverage and CRPS) and with the accuracy–cost frontier as a function of ODE solver and number of steps.

O4 Build a benchmark with an analytic non-Gaussian posterior and use it to measure the fidelity and diversity of the samples.

O5 Document the development in an executable tutorial and release a tested, reproducible repository.

Hypotheses.

H1 Our reimplementation matches the official log-likelihood to within two standard deviations across seeds.

H2 Likelihood and calibration saturate with well under 100 integration steps, while sample quality with 5–10 Euler steps degrades measurably; higher-order solvers reach a given accuracy with fewer steps.

H3 On the cosine benchmark, whose posterior is multimodal, FlowNP recovers the plausible modes better than models whose predictive is Gaussian per target or per autoregressive step.

Scope. Consistency of the model under marginalisation (Kolmogorov consistency), which the architecture does not guarantee [1], is outside the scope of this report and is left as future work.

3 Related Work and State of the Art

Neural processes (NPs) learn a stochastic process directly from a dataset of functions [3, 4]. By Kolmogorov's extension theorem, the predicted joint distributions define a valid process if they are exchangeable and consistent under marginalization; most NPs are exchangeable by construction, but none guarantees every form of consistency [1].

Classical and attentive NPs. The Conditional NP (CNP) [3] predicts an independent Gaussian for each target; the NP [4] adds a global latent variable and the Attentive NP (ANP) [7] adds attention over the context. These models tend to underfit and place most of the uncertainty in per-point variances rather than in coherent samples.

Transformer NPs. The Transformer NP [12] frames prediction as a sequence-modelling task. Its autoregressive variant (TNPA) is the strongest classical baseline, but it generates targets one at a time, uses a Gaussian distribution at every step, and makes the joint distribution dependent on target ordering.

Diffusion NPs. The Neural Diffusion Process (NDP) [2] models joint distributions with diffusion and derives conditionals through likelihood ratios. Because conditioning is not amortized, generating conditional samples requires auxiliary guidance.

Flow matching and FlowNP. Flow matching [9, 10] learns the velocity of a path from Gaussian noise to the data and obtains samples and exact likelihoods by solving an ODE. FlowNP [1] applies it to NPs: the context enters the network directly, so conditioning is amortized and all targets are generated in parallel. The original paper reports state-of-the-art log-likelihoods on 1D Gaussian processes, EMNIST, CelebA and ERA5 (for example 1.69, 1.30 and -0.50 on the RBF, Matérn and periodic kernels), but evaluates only likelihood: the calibration of the uncertainty and the cost of the ODE solver are not assessed, which motivates this study.

4 Methodology

4.1 Problem setting and notation

A function $f : R \to R$ is observed at M context inputs, giving $C = \{(x_i, y_i)\}_{i=1}^{M}$, and the model must predict $y^{\text{tar}} \in R^N$ at target inputs $x^{\text{tar}} \in R^N$. We write $p_\theta(y^{\text{tar}} \mid x^{\text{tar}}, C)$ for the predictive distribution of a model with parameters θ. Every model in this report is trained on batches of functions drawn from a generator, and each batch is stored as a tuple (x^c, y^c, x^t, y^t) of context and target inputs and outputs.

4.2 Flow matching neural process

Training objective. Let $y_1 = y^{\text{tar}}$ be the true targets and $y_0 \sim \mathcal{N}(0, I)$ noise. The linear path is $y_t = t\, y_1 + (1-t)\, y_0$ for $t \in [0, 1]$, whose velocity is $y_1 - y_0$. The network u_θ is trained by regression on this velocity,

$$\mathcal{L}(\theta) = E \left\| u_\theta\big(y_t, t, x^{\text{tar}}, C\big) - (y_1 - y_0) \right\|^2, \tag{1}$$

where the expectation is over functions, context and target sets, $t \sim \mathcal{U}[0, 1]$ and y_0. In our implementation t is drawn independently for each target point by default, with an option to draw one value per function.

Architecture. Figure 1 summarises the network. Each token $[x, t, y]$ is embedded by an MLP after the scalars x and t are encoded with sinusoidal features of frequencies $\pi 2^{j-2}$. Context tokens carry $t = 1$ and their true y; target tokens carry the current time t and the noisy value y_t. All tokens go through a transformer encoder with 6 layers of full self-attention, model dimension 64 and 4 heads, with no positional encoding (we follow the official configuration, which differs from the width of 128 stated in the paper). A head applied only to the target tokens returns the velocity. Since there is no positional encoding, the output is invariant to the ordering of the context and equivariant to the ordering of the targets, so the model is a valid NP for sets. The network has 224 449 parameters.

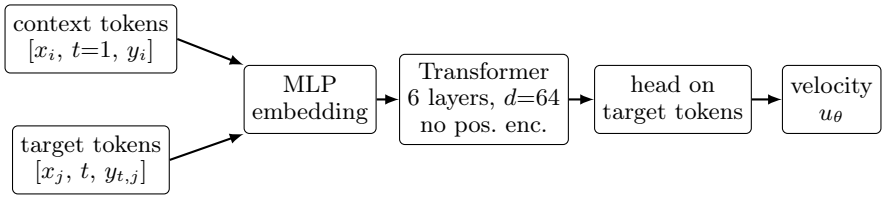


Figure 1: Velocity network of FlowNP. Context and target tokens are processed jointly, and only the target outputs are read out.

Sampling. To draw S samples, the context is replicated S times and the ODE $\mathrm{d}y/\mathrm{d}t = u_\theta(y, t, x^{\mathrm{tar}}, C)$ is integrated from $y_0 \sim \mathcal{N}(0, I)$ at $t = 0$ to $t = 1$ for all targets in parallel. With K steps of size $h = 1/K$ we support three explicit solvers: Euler, midpoint and fourth-order Runge–Kutta (RK4), which need 1, 2 and 4 network evaluations per step respectively, so NFE $= K$, $2K$ and $4K$.

Likelihood. By the change-of-variables formula for continuous flows,

$$\log p_\theta(y_1 \mid x^{\mathrm{tar}}, C) = \log \mathcal{N}(y_0; 0, I) - \int_0^1 \operatorname{tr}\left(\frac{\partial u_\theta}{\partial y}\right) \mathrm{d}t, \tag{2}$$

where y_0 is obtained by integrating the ODE backwards from $t = 1$ to $t = 0$ starting at y_1. The divergence is computed either *exactly*, with one backward pass per target coordinate, or with the Hutchinson estimator [6, 5] $\operatorname{tr}(J) = E_\epsilon[\epsilon^\top J \epsilon]$ with Rademacher vectors $\epsilon \in \{\pm 1\}^N$, which needs a single backward pass. We report the log-likelihood divided by the number of targets.

4.3 Implementation and verification

The code is organised in a package (`flownp`) with the flow core (loss, sampling, divergence and likelihood), the velocity network, a common interface and the training and evaluation routines. We reimplement FlowNP from the paper rather than copying the official code. The official implementation is loaded dynamically at a pinned commit (`f3403b6`) to build the reference models.

Common predictor interface. A wrapper exposes `samples`, `log_likelihood` and `loss` for every model (FlowNP, official FNP, NDP, CNP, ANP, TNPA) and hides their differences. For ANP the likelihood is a log-sum-exp over 200 latent samples; for TNPA the targets are randomly permuted before the autoregressive pass and the permutation is undone afterwards; NDP requires gradients to be enabled.

Verification. We check the reimplementation in three ways. (i) *Architecture:* the keys of an official FNP checkpoint are renamed (`embedder.`→`embed.`, `predictor.`→`head.`) and loaded into our network, which only succeeds if both architectures coincide. (ii) *Unit tests* (13) cover the likelihood against an analytic Gaussian with exact and Hutchinson divergence, the fact that Euler overestimates the likelihood but converges as steps increase, sampling against the target distribution, permutation symmetry, agreement of Hutchinson with the exact divergence in the transformer, the decrease of the loss during training, the cosine posterior and its classifier, exact resumption of training, and the metrics (which must reward

the exact posterior and penalise blurred samples). (iii) *Numerical comparison* with the official implementation under the same data and budget (H1).

Training. All models are trained with Adam [8] with cosine annealing of the learning rate [11], and share the same iteration budget. Checkpoints are written atomically and contain the model, optimiser, scheduler, random state and configuration, so an interrupted run yields the same result as an uninterrupted one. Each run also writes a metadata file with the commit, library versions, GPU and number of parameters.

4.4 Data, protocol and metrics

Gaussian processes. RBF, Matérn-5/2 and periodic kernels with hyper-parameters drawn per function, $x \in [-2, 2]$, 3–47 context points and up to 50 points in total, as in the original paper; each kernel has a fixed evaluation set of 100×128 functions shared by all models.

Cosines. $y(x) = a\cos(2\pi fx) + b\sin(2\pi fx) + \varepsilon$ with $f \in \{0.25, 0.5, 0.75, 1\}$, $(a, b) \sim \mathcal{N}(0, I)$ and $\varepsilon \sim \mathcal{N}(0, 0.05^2)$. Given f the function values are jointly Gaussian, so the exact posterior is a mixture of four Gaussians and is multimodal when the context is small. Training uses 1–20 context points; evaluation uses $M \in \{1, 2, 3, 5, 10, 20\}$ and a 100-point grid.

Protocol. Baselines and the official FlowNP are trained with the official code and configuration (Adam, learning rate $5 \cdot 10^{-4}$ with cosine decay, batches of 16 functions, 10^5 steps); 5 seeds per model (3 for NDP). On cosines every model starts from its own RBF checkpoint and is trained for 10^5 steps with 3 seeds; FlowNP is also trained from scratch. All runs used Google Colab T4 GPUs.

Metrics. Log-likelihood per target point; coverage of the central 90% interval and CRPS from 64 samples per task; on cosines, *fidelity* (fraction of samples whose best cosine fit has error below $2\sigma_\varepsilon$ and non-negligible amplitude) and the total-variation (TV) distance between the frequencies of the valid samples and the exact posterior.

5 Results and Discussion

5.1 Results on Gaussian Processes

We compare FlowNP (FNP) with the Conditional Neural Process (CNP), the Attentive Neural Process (ANP), the autoregressive Transformer Neural Process (TNPA) and the Neural Diffusion Process (NDP). The metric is the mean log-likelihood per target point, where higher values are better.

Table 1: Mean target log-likelihood and standard deviation across random seeds for each Gaussian-process kernel (3 seeds for NDP, 5 for the others).

Kernel	ANP	CNP	FNP	NDP	TNPA
RBF	1.1545 ± 0.0062	0.3274 ± 0.0138	$\mathbf{1.7161 \pm 0.0037}$	1.2032 ± 0.0078	1.6881 ± 0.0040
Matérn	0.8965 ± 0.0104	0.1273 ± 0.0129	$\mathbf{1.3342 \pm 0.0023}$	0.9118 ± 0.0087	1.3212 ± 0.0049
Periodic	-0.8519 ± 0.0245	-0.6234 ± 0.0023	-0.4984 ± 0.0007	-0.5420 ± 0.0037	-0.5722 ± 0.0004

On all three kernels FNP achieves the highest log-likelihood, as reported in the original paper. The gap to TNPA is small on RBF and Matérn, and on the periodic kernel FNP also clearly outperforms NDP.

5.2 Reproduction of the RBF Result

Hypothesis H1 compares our FlowNP reimplementation with the official result on the RBF kernel. Across five seeds our reimplementation obtains $\mathbf{1.7119 \pm 0.0031}$, against $\mathbf{1.7161 \pm 0.0037}$ for the official model. The difference is within the pre-registered margin of two

standard deviations and is not statistically significant (Welch's t-test, $p \approx 0.11$). Loading the official weights into our model gives the same velocities up to $2 \cdot 10^{-6}$, so both implementations are numerically equivalent, which supports H1.

5.3 Calibration of the Predictive Uncertainty

Log-likelihood does not show whether the spread of the samples is appropriate. We therefore measured, on the official checkpoints and with 64 samples per task, the empirical coverage of the central 90% interval (ideal value 0.90) and the continuous ranked probability score (CRPS, lower is better). FlowNP uses the Euler sampler with 100 steps of the paper; we also evaluated the stochastic sampler shipped with the official code (Appendix C of the original paper), which rescales the velocity by $1 + t(1 - t)$ and adds small noise.

Table 2: Coverage of the 90% interval and CRPS on the Gaussian-process kernels (mean over 5 seeds; standard deviations ≤ 0.01 for coverage and ≤ 0.002 for CRPS).

	RBF		Matérn		Periodic	
Model	Cov. 90	CRPS	Cov. 90	CRPS	Cov. 90	CRPS
CNP	0.857	0.121	0.848	0.139	0.845	0.295
ANP	0.931	0.072	0.912	0.090	0.737	0.319
TNPA	**0.884**	**0.064**	**0.877**	**0.083**	0.879	0.289
FNP (Euler, 100 steps)	0.832	**0.064**	0.842	0.084	**0.880**	**0.287**
FNP (official stochastic sampler)	0.599	0.067	0.607	0.088	0.801	0.289
FNP (ours)	0.834	**0.064**	–	–	–	–

As Table 2 shows, FlowNP is as sharp as TNPA in CRPS but mildly over-confident: its 90% intervals contain the true value only 83–84% of the time on RBF and Matérn, against 88% for TNPA. The stochastic sampler of the official code is badly calibrated, with 60% coverage, although its CRPS barely changes (0.064 to 0.067); CRPS alone is therefore insensitive to this kind of under-dispersion. Our reimplementation shows the same calibration as the official model. The diffusion model is not included on RBF and Matérn because its guided sampler ran out of GPU memory.

5.4 Solver Accuracy and Sampling Cost

FlowNP generates samples and computes likelihoods by solving an ODE, so quality and cost depend on the solver and on the number of steps. We evaluated the official FlowNP checkpoints on RBF (seeds 0–2) with Euler, midpoint and fourth-order Runge–Kutta (RK4) for 5 to 100 steps. The likelihood uses the first 20 evaluation batches, so its absolute value differs from Table 1.

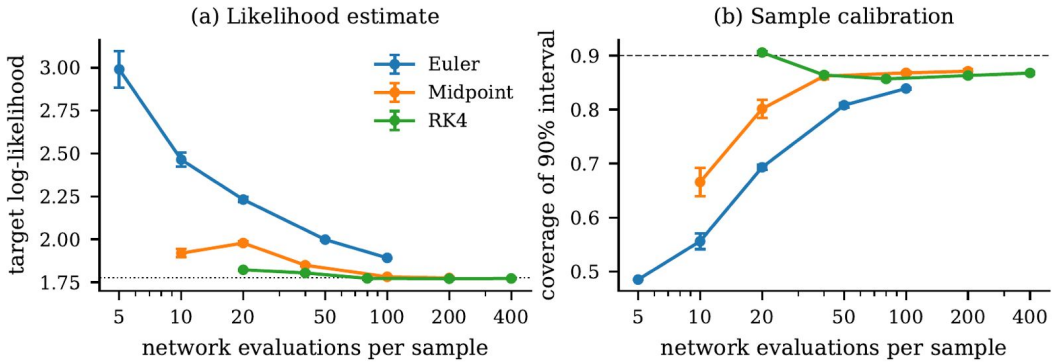


Figure 2: Official FlowNP on RBF (3 seeds): likelihood estimate (dotted line: converged value) and coverage of the 90% interval as a function of the network evaluations per sample.

Figure 2 shows that Euler strongly over-estimates the likelihood (2.99 with 5 steps and still 1.89 with 100, against a converged value of about 1.77), which is consistent with Appendix D of the original paper. Midpoint and RK4 converge within 20–50 steps, which supports hypothesis H2. For sampling, Euler with 5 steps covers only 49% of the values and reaches 84% with 100 steps, whereas the converged solvers stabilise at 86–87%: part of the over-confidence of FlowNP is discretisation error and the remainder belongs to the learned flow. RK4 with only 5 steps (20 network evaluations) reaches 91% coverage in 45 s, against 84% in 230 s for 100-step Euler, so it is a better default.

5.5 Fidelity and Diversity in the Cosine Experiment

The cosine benchmark tests whether a model generates valid samples and covers the plausible frequencies when the context does not determine a unique answer, using the exact posterior as ground truth.

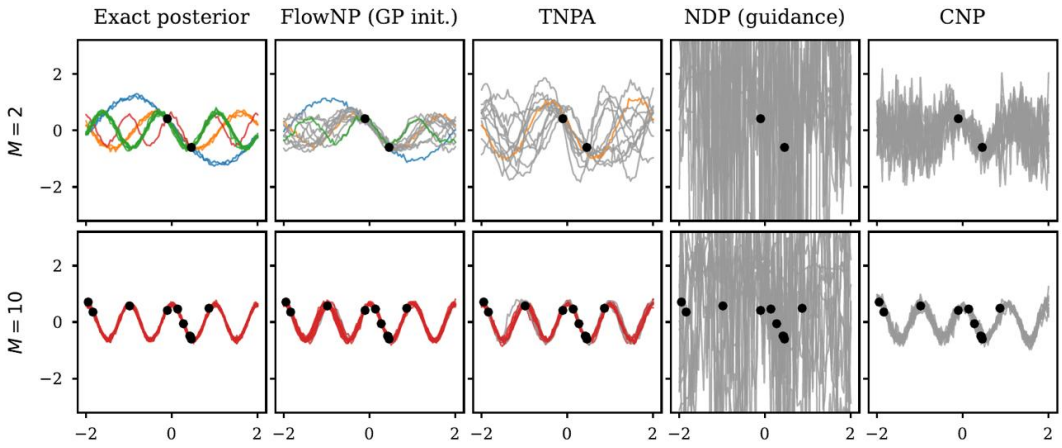


Figure 3: Twelve samples on a 100-point grid for one cosine task (true frequency 1), conditioned on its first M points (black), seed 0 of each model. Colour: frequency of the best cosine fit (blue 0.25, orange 0.5, green 0.75, red 1); grey: not a valid cosine. With $M = 2$ the exact posterior mixes three frequencies; FlowNP recovers two of them, TNPA mostly produces invalid curves, CNP produces independent noise, and the guided sampler of NDP diverges on this task (it does so on a fraction of tasks). With $M = 10$ FlowNP and TNPA match the exact posterior.

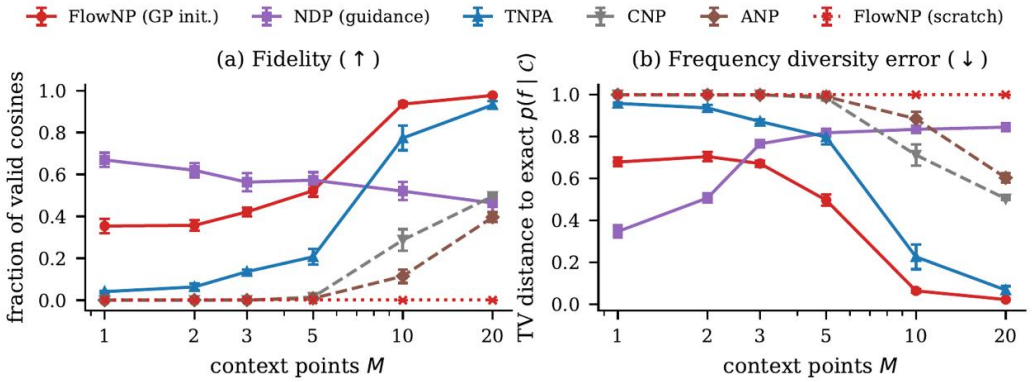


Figure 4: Cosine benchmark: fidelity and frequency-diversity error against the exact posterior as a function of the number of context points (mean ± standard deviation over 3 seeds).

Table 3: Cosine benchmark (mean over 3 seeds). The exact posterior log-likelihood is 1.482. Fidelity: fraction of samples that are valid cosines with M context points. TV: total-variation distance between the generated frequencies and the exact posterior (lower is better).

Model	Log-lik.	Cov. 90	Fid. M=1	TV M=1	Fid. M=20	TV M=20
CNP	0.388	0.86	0.00	1.00	0.50	0.50
ANP	0.392	0.85	0.00	1.00	0.40	0.60
NDP	**1.372**	0.63	**0.67**	**0.35**	0.46	0.85
TNPA	1.204	**0.91**	0.04	0.96	0.93	0.07
FNP (GP init.)	1.275	0.85	0.35	0.68	**0.98**	**0.02**
FNP (from scratch)	−0.833	0.90	0.00	1.00	0.00	1.00

Figure 3 illustrates the samples of each model and Figure 4 and Table 3 summarise all models. With informative contexts FlowNP is the most faithful model: with 20 points, 98% of its samples are cosines of the correct frequency, ahead of TNPA (93%). With a single context point, however, only 35% of its samples are valid cosines, and the diffusion model covers the plausible frequencies better (TV 0.35 against 0.68). The diffusion model obtains the highest likelihood, computed as a ratio of joint likelihoods, but its guided samples increasingly ignore the context (TV 0.85 with 20 points and 63% coverage). CNP and ANP are reasonably calibrated (86% and 85%) yet produce almost no valid curves, since independent noise at each point yields jagged functions: calibration does not imply fidelity. Finally, FlowNP trained from scratch stays in a plateau where it ignores the context, whereas TNPA trained in the same way learns without difficulty.

Hypothesis H3 is supported with respect to the models with Gaussian predictives: with one context point FlowNP recovers the plausible frequencies better than TNPA, CNP and ANP (TV 0.68 against 0.96–1.00), and with 20 points it is the closest to the exact posterior (TV 0.02 against 0.07 for TNPA). The diffusion model, whose predictive is not Gaussian, covers the modes better than FlowNP when the context is very small. TNPA remains the best calibrated model.

6 Conclusions and Future Work

Overall, the results suggest that FlowNP performs competitively on regression tasks involving synthetic functions. Our reimplementation achieved an RBF log-likelihood close to that of the official model. The multimodal cosine experiment also highlights the importance of initialization: in the reported runs, training from scratch did not yield reliable cosine samples, whereas initialization from a pretrained model produced substantially better results, especially with more context points.

The solver sweep shows that fourth-order Runge–Kutta with 5 steps offers a better trade-off between calibration and cost than the default 100-step Euler sampler, and the calibration study shows that FlowNP is mildly over-confident and that the stochastic sampler of the official code should be avoided when uncertainty matters.

These findings are limited to one-dimensional synthetic functions. They demonstrate modeling distributions over curves, but do not evaluate image or digit generation. In addition, calibration of the diffusion model on the RBF and Matérn kernels could not be completed because of GPU memory limitations, so the diffusion comparison is incomplete. Future work includes testing marginal consistency, extending the study to images and weather data, and understanding why FlowNP trained from scratch ignores the context on multimodal data.

References

[1] Hussen Abu Hamad and Dan Rosenbaum. Flow matching neural processes. *Advances in Neural Information Processing Systems*, 38:116498–116521, 2026.

[2] Vincent Dutordoir, Alan Saul, Zoubin Ghahramani, and Fergus Simpson. Neural diffusion processes. In *International Conference on Machine Learning*, pages 8990–9012. PMLR, 2023.

[3] Marta Garnelo, Dan Rosenbaum, Christopher Maddison, Tiago Ramalho, David Saxton, Murray Shanahan, Yee Whye Teh, Danilo Rezende, and SM Ali Eslami. Conditional neural processes. In *International conference on machine learning*, pages 1704–1713. PMLR, 2018.

[4] Marta Garnelo, Jonathan Schwarz, Dan Rosenbaum, Fabio Viola, Danilo J Rezende, SM Eslami, and Yee Whye Teh. Neural processes. *arXiv preprint arXiv:1807.01622*, 2018.

[5] Will Grathwohl, Ricky TQ Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. Ffjord: Free-form continuous dynamics for scalable reversible generative models. *arXiv preprint arXiv:1810.01367*, 2018.

[6] Michael F Hutchinson. A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines. *Communications in Statistics-Simulation and Computation*, 18(3):1059–1076, 1989.

[7] Hyunjik Kim, Andriy Mnih, Jonathan Schwarz, Marta Garnelo, Ali Eslami, Dan Rosenbaum, Oriol Vinyals, and Yee Whye Teh. Attentive neural processes. *arXiv preprint arXiv:1901.05761*, 2019.

[8] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[9] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.

[10] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. *arXiv preprint arXiv:2209.03003*, 2022.

[11] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. *arXiv preprint arXiv:1608.03983*, 2016.

[12] Tung Nguyen and Aditya Grover. Transformer neural processes: Uncertainty-aware meta learning via sequence modeling. *arXiv preprint arXiv:2207.04179*, 2022.

A Reproducibility and hyper-parameters

All code, configurations, tests and logs are available in the public repository `https://github.com/samuel13032004/flownp-repro`, together with an executable tutorial (`https://samuel13032004.github.io/flownp-repro/`). All runs are launched from two Colab notebooks: `notebooks/colab_referencias.ipynb` for the official baselines and `notebooks/colab_experimentos.ipynb` for our experiments. The official code is pinned to commit `f3403b6` and is cloned separately because it carries no licence. Evaluation sets are generated once with seed 0 on CPU and shared by all models.

Two official runs (ANP and TNPA, RBF, seed 3) were resumed after a session cut; since the official checkpoints do not store the random state, their training stream differs from an uninterrupted run. Two of our runs (RBF, seeds 0 and 1) were resumed with our training loop, which restores the random state exactly.

Table 4: Main hyper-parameters of our FlowNP reimplementation (identical to the official configuration).

Hyper-parameter	Value
Transformer layers / heads / model dimension	6 / 4 / 64
Positional encoding	none
Number of parameters	224 449
Optimiser	Adam, learning rate $5 \cdot 10^{-4}$, cosine decay
Batch size	16 functions
Training steps	10^5
Flow time t	$\mathcal{U}[0, 1]$, one value per target point
Default sampler	Euler, 100 steps
Samples per task (calibration)	64
Hardware	Google Colab, NVIDIA T4 GPU

B Solver sweep

Table 5 lists the values plotted in Figure 2. Likelihoods use the first 20 evaluation batches, so their absolute values differ from Table 1; calibration uses 5 batches and 64 samples. On Matérn-5/2 the pattern is the same (coverage 0.85 for 100-step Euler, 0.89 for 5-step RK4); on the periodic kernel all solvers reach 0.88 with 5–10 steps.

Table 5: Solver sweep for the official FlowNP on RBF (mean over 3 seeds). NFE: network evaluations per sample. Time: sampling time for the calibration set.

Solver	Steps	NFE	Log-lik.	Cov. 90	Time (s)
Euler	5	5	2.990	0.485	12
Euler	20	20	2.232	0.693	47
Euler	100	100	1.892	0.839	230
Midpoint	10	20	1.978	0.801	35
Midpoint	100	200	1.775	0.871	426
RK4	5	20	1.822	**0.906**	45
RK4	20	80	1.772	0.857	178
RK4	100	400	1.772	0.868	892

C Exact posterior of the cosine benchmark

For a fixed frequency f, a function is $y(x) = \phi_f(x)^\top w + \varepsilon$ with $\phi_f(x) = [\cos(2\pi f x), \sin(2\pi f x)]^\top$, $w = (a, b) \sim \mathcal{N}(0, I)$ and $\varepsilon \sim \mathcal{N}(0, \sigma_\varepsilon^2)$, $\sigma_\varepsilon = 0.05$. The values at any set of inputs are therefore jointly Gaussian with zero mean and covariance $K_f(x, x') = \phi_f(x)^\top \phi_f(x') + \sigma_\varepsilon^2 \, \delta_{xx'}$. Given a context C, Bayes' rule with a uniform prior

over the four frequencies gives

$$p(f \mid C) = \frac{\mathcal{N}(y^c; 0, K_f(x^c, x^c))}{\sum_{f'} \mathcal{N}(y^c; 0, K_{f'}(x^c, x^c))}, \tag{3}$$

and the predictive distribution is the mixture

$$p(y^{\text{tar}} \mid x^{\text{tar}}, C) = \sum_f p(f \mid C)\, \mathcal{N}\left(y^{\text{tar}}; \mu_f, \Sigma_f\right), \tag{4}$$

where μ_f and Σ_f are the usual Gaussian conditional mean and covariance under K_f. This mixture gives the exact reference log-likelihood (1.482 in Table 3), exact samples, and the exact frequency probabilities used in the TV distance. With few context points several frequencies keep non-negligible probability and the posterior is multimodal.

D Metric details and further observations

Cosine fidelity. Each sample on the 100-point grid is fitted by least squares with a cosine of each candidate frequency; the best fit gives its frequency and amplitude. A sample is valid if the root-mean-square residual is below $2\sigma_\varepsilon = 0.1$ and the fitted amplitude is above 0.1 (a flat line otherwise fits every frequency with zero amplitude). The TV distance compares the frequencies of the samples with $p(f \mid C)$; invalid samples are counted as error. Each value of M uses 64 tasks and 64 samples per task.

Missing entries. NDP is absent from Table 2 on RBF and Matérn-5/2 because its guided sampler, which back-propagates through the network for all 64×128 samples of a batch, ran out of GPU memory. On the periodic kernel two of three seeds completed (coverage of the 90% interval 0.98, CRPS 0.38), so it is strongly under-confident there.

Other observations. (i) The official TNPA sampler returns samples in its internal permuted order rather than in the order of the requested inputs, which produces jagged curves; we undo the permutation explicitly. (ii) CNP and ANP likelihoods are per-point marginals, whereas flow and autoregressive models report joint likelihoods divided by the number of targets, so the two are not strictly comparable. (iii) FlowNP trained from scratch on cosines stays in a plateau whose loss is close to that of a velocity field that ignores the context: reversing the sign of the context values changes its predicted velocities by only 0.03. Initialising from the RBF checkpoint removes the plateau.

E Computational cost

Table 6 reports the GPU time used in the project. All runs were executed on Google Colab with an NVIDIA T4 GPU, and the total is about 90 GPU hours.

Table 6: GPU hours used by each stage of the project.

Stage	GPU hours
Reference models: training with the official code (3 kernels × 5 models, 5 seeds, or 3 for the diffusion model)	51.8
Reference models: evaluation (official code)	3.9
Our FlowNP: reproduction training (5 seeds)	4.0
Cosine benchmark: training (6 variants × 3 seeds)	13.5
Our evaluations (calibration, solvers, cosines)	16.6
Total	89.8

Most of the cost comes from the baselines (about 62% of the total, between training and evaluation), while our reimplementation and its experiments take the remaining 38%.

Author Contributions

Following the CRediT taxonomy. **Yaser Jafar**: Software, Investigation, Data curation, Visualization, Writing – review & editing. **Samuel Déniz**: Writing – original draft, Formal analysis, Software (repository and documentation site), Project administration. **Carlos Enrique Pereda**: Writing – original draft, Formal analysis, Validation, Writing – review & editing. All authors contributed to Conceptualization and Methodology and approved the final manuscript.

Declaration Regarding the Use of AI Tools

During the preparation of this manuscript, the authors used Claude (Anthropic, Claude Opus 5.5) for coding, experiment design, literature support and partial support in writing the report. In particular, the tool helped write and test the reimplementation, the experiment notebooks and the tutorial, propose the evaluation protocol, and draft and polish some parts of the report from the authors' results. The authors independently verified the output and retain full responsibility for the accuracy, integrity, originality, and final content of the manuscript.